

Prompts we used to code the application using Gemini 3.1 Pro in the Google Antigravity IDE:

Chat Conversation

Note: This is purely the output of the chat conversation and does not contain any raw data, codebase snippets, etc. used to generate the output.

User Input

Write a simple Python program that has a GUI interface made using TKInter that has a window where users can enter text that connects to a Google Gemini model using the API (I have a key if needed), and it will show the models responses

Edited relevant file

User accepted the command ``pip install google-generativeai; python gemini\_gui.py``

Planner Response

I have created ``gemini\_gui.py`` for you in the current directory. This is a standalone script that provides a graphical user interface created with TKInter allowing you to chat with a Gemini model.

What it does:

API Key Field: It gives you a text field at the very top of the window to enter your Gemini API key (it hides the characters to keep it secure).

Chat Area: It has a scrollable text area for the ongoing conversation, with distinct formatting for "You", "Gemini", and "System Errors".

Responsive Execution: The tool uses threads when talking to the Gemini API so the GUI will not freeze while the model is "thinking".

Convenience: You can type out multi-line messages, and hitting ``Enter`` sends the message (``Shift+Enter`` moves to a new line).

How to use it:

I've queued up a terminal command for you to run. Once you approve it, it will achieve two things:

It will install ``google-generativeai`` if you don't already have it installed.

It will start the newly created application ``gemini\_gui.py``.

Once the window pops up, simply paste your Google Gemini API key into the top field, type your message, and hit Send! Let me know if you want to make any adjustments.

User Input

Alright this works great, I want to now add a message in the chat that says "chat message sent to Gemini API, please wait a few seconds..." that appears after the user sends a message, and then, if possible, disappears and is replaced by the models response.

``*Viewed gemini\_gui.py``

Edited relevant file

Planner Response

I have updated the ``gemini\_gui.py`` program to include the waiting message!

Now, immediately after you hit send, the chat window will display:

``> \*System: chat message sent to Gemini API, please wait a few seconds...\*``

Once the response (or an error) is received from the Gemini API, that temporary waiting message will be deleted, and the actual response from Gemini will take its place.

You can try it out by running the program again! Let me know if you would like me to make any adjustments.

User Input

Now it's time to add the main function of the app. Our app is a simple app for a class final project to detect and explain misinformation.

I want to keep basic chat setup we have now, but I want to add a textbox under the Gemini API key box but above the main chat box that will display a verdict on whether a given statement is a true statement or a statement of misinformation.

Here are the possible outcomes that we'll go with for now:

No statement given: The user's chat input did not contain an statement to verify as true or as misinformation.

True: The statement given by the user is true.

False: The statement given by the user is false. (Misinformation goes here)

Needs context: The statement given by the user is such that it should not be marked true or false, as it: blends true statements with false/misinformed statements; is true but it was taken out of relevant context that is essential to understanding it correctly; or the like.

Unknowable: Nobody can say whether it is true or false.

Error: Gemini did not respond in a way that could be processed by the application, as per the instructions given to it (processing detailed below.)

Here are the box colors and text colors I would like the text box to show given the following verdict outcomes:

No statement given: White box with black text that says "No Statement Given"

True: Dark Green box with white text that says "True"

False: Dark red box with white text that says "False"

Needs context: Yellow box with black text that says "Needs Context"

Unknowable: Dark purple box with white text that says "Unknown"

Now we need a way to get the Gemini API to respond in a way that is suitable for our application, and can be processed by it to show which option it grouped the statement in so that we can update the textbox that shows its verdict.

If there is a way to give it a prompt for every conversation, then give it this prompt:

"Your task is to take the statement given by the user and classify it based on whether it is a true statement or a false statement/statement of misinformation.

Here are the five verdict categories you will choose from when classifying the user's statement:

No statement given: Choose this verdict if the user's chat input did not contain an statement to verify as true or as misinformation. (If this occurs, respond as normal, but ensure that whatever you say, put "Verdict: No Statement Given" at the end of your response.)

True: The statement given by the user is true.

False: The statement given by the user is false. (Choose this is the statement is misinformation and therefore false)

Needs context: The statement given by the user is such that it should not be marked true or false, as it: blends true statements with false/misinformed statements; is true but it was taken out of relevant context that is essential to understanding it correctly; or the like.

Unknowable: Nobody can say whether it is true or false.

Respond in the following format:

"Explanation: <Give an explanation that explains whether this statement is true, false, needs context, or unknowable.>

Verdict: <Respond "True", "False", "Needs Context", or "Unknowable", e.g. "Verdict: True">

I will leave it to you to find the most prudent way of prompting the LLM API and choosing the most prudent logic to interpret the LLM's response and process it into one of the six possible outcomes.

Edited relevant file

Planner Response

I have updated the app with the requested main functionality!

What was added:

Verdict Banner: There is now a label right underneath the API key box. It defaults to a white background with the text "No Statement Given".

System Prompt Configuration: The Gemini model is now initialized with the strict system instruction you provided. It understands its role as a misinformation detection tool and categorizes your messages into one of the 5 categories. It will return an "Explanation" along with a "Verdict" format.

Verdict Parser: Once the API responds, the chat interface extracts the word following `Verdict:` from the model's text. It then matches it to your specified requirements and updates the top banner to reflect that outcome.

If the verdict is True, it turns the banner Dark Green with white text.

If the verdict is False, it turns the banner Dark Red with white text.

If it Needs context, the banner goes Yellow with black text.

If it's Unknowable/Unknown, the banner becomes Purple with white text.

If the user had no statement to verify, it turns White with black text.

An Error (if the system prompt isn't followed for some reason) makes the banner Gray with white text.

Run the app again using python to check it out. You can enter something like "The earth is flat" and after the model processes the response, the banner should turn Dark Red and say "False".

Let me know if you would like me to process the output any differently!